RTOS_TIMER

开发指南

版本号: 1.6

发布日期: 2026.01.15

深圳易新泰科技有限公司YiXinTai

版本历史

版本号	日期	制/修订人	内容描述
1.0	2021.04.11	AWA1637	初始版本
1.1	2023.02.15	AWX0249	修改格式
1.2	2024.03.07	AWX0315	添加 AI985 的说明
1.3	2025.03.31	AW0339	1. 补充使用范围 2. 添加 Timer 和 HSTimer 的接口使用说明
1.4	2025.06.23	AWA1615	添加 F135,F136 说明
1.5	2025.08.27	XAA0368	修改全文格式
1.6	2026.01.15	XAA0331	添加 F101,F102 说明

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

目录

第 1 章 前言 1
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1
1.4 文档约定 1
1.4.1 符号约定 1
1.4.2 地址与数据描述约定 2
1.4.3 数值单位约定 2
1.5 术语说明 2
1.5.1 硬件术语 2
1.5.2 软件术语 2

第 2 章 模块介绍 3
2.1 模块功能介绍 3
2.1.1 TIMER 3
2.1.2 HSTIMER 3
2.2 模块寄存器配置介绍 3
2.3 menuconfig 配置介绍 4
2.3.1 TIMER 模块 4
2.3.2 HSTIMER 模块 5
2.4 模块源码结构 5
2.4.1 TIMER 模块源码结构 5
2.4.2 HSTIMER 模块源码结构 5
2.5 软件框架 6

第 3 章 模块接口说明 7
3.1 接口列表 7
3.1.1 TIMER 接口 7
3.1.2 HSTIMER 接口 7
3.2 接口使用说明 7
3.2.1 Timer 初始化接口 7
3.2.2 Timer 失能接口 7
3.2.3 Timer 开启计数接口 7
3.2.4 Timer 关闭计数接口 8
3.2.5 Timer 周期性定时功能 8
3.2.6 Timer 单次定时功能 8
3.2.7 HSTimer 基础时钟初始化接口 8
3.2.8 HSTimer 基础时钟反初始化接口 8
3.2.9 HSTimer 计数器读取接口 8

第 4 章 模块使用范例 9
4.1 TIMER 模块使用范例 9
4.2 HSTIMER 模块使用范例 10

第 5 章 调试方法 14
5.1 TIMER 测试方法 14
5.2 HSTIMER 测试方法 14

第 6 章 FAQ 15

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

第 1 章 前言

1.1 文档简介

介绍 RTOS 中 Timer 驱动及 HTimer 的接口及使用方法，为 Timer 使用者提供参考。

1.2 目标读者

Timer 驱动层/应用层开发/使用/维护人员。

1.3 适用范围

表 1-1: 适用产品列表

产品名称	内核版本	驱动文件
V459	Melis	hal_timer.c
F133	Melis	hal_timer.c
F135	Melis	hal_timer.c
F136	Melis	hal_timer.c
R328	FreeRTOS	hal_timer.c
R128	FreeRTOS	hal_timer.c
XR875	FreeRTOS	hal_timer.c
MR527	FreeRTOS	hal_timer.c
AI985	FreeRTOS	hal_htimer.c
MR153	FreeRTOS	hal_htimer.c
T153	FreeRTOS	hal_htimer.c
F101	Melis	hal_htimer.c
F102	Melis	hal_htimer.c

1.4 文档约定

本节规定了本文档在技术描述中所遵循的表示法、单位与符号体系，是理解后续内容的基础规范。

1.4.1 符号约定

本文档中会使用不同符号区分信息类型，具体释义如下：



说明

用于呈现技术核心信息（如功能原理、核心参数定义）、流程补充内容（如步骤细节说明）等。



技巧

用于分享技术操作中的高效方法（如命令行快捷指令、配置项优化窍门）与实践技巧。



注意

用于突出技术操作中易出错的环节（如参数配置边界、操作顺序要求）的提示。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

1.4.2 地址与数据描述约定

在描述寄存器地址、配置值、数据格式及位域时，采用如下符号体系：

表 1-2: 地址与数据描述约定

符号	示例	说明
0x	0x0200，0x79	地址或数据采用 16 进制数
0b	0b010，0b00 000 111	数据采用二进制数（寄存器描述除外）
X	00X，XX1	通配符，表示该位可取 0 / 1 例：00X 表示 000/001

1.4.3 数值单位约定

不同物理量遵循特定的单位换算标准，具体如下：

表 1-3: 数值单位约定

类型	单位示例	换算基准	对应数值
数据容量	1 KB 1 MB 1 GB	以 1024 为基数	1024 Byte 1 048 576 Byte 1 073 741 824 Byte
频率、数据速率	1 K Hz 1 M Hz 1 G Hz	以 1000 为基数	1000 Hz 1 000 000 Hz 1 000 000 000 Hz
数据位宽	1 bit 1 Byte 1 Word	1 Byte = 8 bit 1 Word = 32 bit	1 bit 8 bit 32 bit

1.5 术语说明

1.5.1 硬件术语

表 1-4: 硬件术语

术语	解释说明
Timer	定时器
HSTimer	高精度定时器

1.5.2 软件术语

表 1-5: 软件术语

术语	解释说明
HAL	Hardware Abstraction Layer，硬件抽象层

版权所有 © 珠海全志科技股份有限公司。保留一切权利

第 2 章 模块介绍

2.1 模块功能介绍

2.1.1 TIMER

Timer 模块可以用于实现计时、计数功能。其具体规格如下所示：

- 计数时钟可配置:LOSC 和 OSC24M
- 可配置 8 种预分频系统
- 可编程 32-bit 减法定时计数器
- 两种工作模式：循环模式和单次计数模式
- 当计数值减到 0 时可以产生中断

2.1.2 HSTIMER

HTimer 模块可以用于实现计时、计数功能。其具体规格如下所示：

- 计数时钟可配置: APB、HOSC24M...
- 可配置 8 种预分频系统
- 可编程 56-bit 减法计数器
- 两种工作模式：循环模式和单次计数模式
- 当计数值减到 0 时可以产生中断

2.2 模块寄存器配置介绍

Timer 和 HSTimer 模块寄存器的基本配置位于文件 timer_{platform}.h 里面，包括每个 Timer 的寄存器地址和中断号，如 timer_sun8iw20.h, 配置如下：

```
/* timer */
#define SUNXI_TMR_PBASE 0x02050000

#if defined(CONFIG_ARCH_ARM) /* ARM */
#define SUNXI_IRQ_TMR(id) (91 + (id))
#elif defined(CONFIG_ARCH_RISCV) /* RISC-V */
#define SUNXI_IRQ_TMR(id) (75 + (id))
#elif defined(CONFIG_ARCH_DSP) /* DSP */
#define SUNXI_IRQ_TMR(id) (21 + (id))
#endif

#define SUNXI_AVS_CLK_TYPE HAL_SUNXI_CCU
#define SUNXI_AVS_CLK   CLK_AVS

/* htimer */
#define SUNXI_SYS_HTIMER_BASE   0x03008000
#define SUNXI_SYS_TIMER_NUM     2
#define HTIMER_MAX_NUM          2

#define SUNXI_HTIMER_BASE(id) SUNXI_SYS_HTIMER_BASE
#define HTIMER_IRQ_EN(id)       BIT(id)
#define PENDING_BIT(id)         BIT(id)

#define HTIMER_CTL_REG_OFFSET(id)       (0x20 * (id) + 0x20)
#define HTIMER_INTVAL_LO_REG_OFFSET(id) (0x20 * (id) + 0x24)
#define HTIMER_INTVAL_HI_REG_OFFSET(id) (0x20 * (id) + 0x28)
#define HTIMER_CNTVAL_LO_REG_OFFSET(id) (0x20 * (id) + 0x2c)
#define HTIMER_CNTVAL_HI_REG_OFFSET(id) (0x20 * (id) + 0x30)

#define TIMER0_PARAMS \
{       .id = 0, \
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

```
        .reg_base = SUNXI_SYS_HTIMER_BASE, \
        .irq_num = SUNXI_IRQ_HSTIMER0, \
        .clk_type = HAL_SUNXI_CCU, .clk_id = CLK_BUS_HSTIMER, \
        .bus_type = HAL_SUNXI_CCU, .bus_id = CLK_BUS_HSTIMER, \
        .reset_type = HAL_SUNXI_RESET, .reset_id = RST_BUS_HSTIMER, \
}

#define TIMER1_PARAMS \
{       .id = 1, \
        .reg_base = SUNXI_SYS_HTIMER_BASE, \
        .irq_num = SUNXI_IRQ_HSTIMER1, \
        .clk_type = HAL_SUNXI_CCU, .clk_id = CLK_BUS_HSTIMER, \
        .bus_type = HAL_SUNXI_CCU, .bus_id = CLK_BUS_HSTIMER, \
        .reset_type = HAL_SUNXI_RESET, .reset_id = RST_BUS_HSTIMER, \
}

#if defined(CONFIG_ARCH_ARM)
#define SUNXI_IRQ_HSTIMER0      87
#define SUNXI_IRQ_HSTIMER1      88
#elif defined(CONFIG_ARCH_RISCV)
#define SUNXI_IRQ_HSTIMER0      71
#define SUNXI_IRQ_HSTIMER1      72
#elif defined(CONFIG_ARCH_DSP)
#define SUNXI_IRQ_HSTIMER0      15
#define SUNXI_IRQ_HSTIMER1      16
#endif
```

2.3 menuconfig 配置介绍

2.3.1 TIMER 模块

需要打开 “CONFIG_DRIVERS_TIMER” 宏定义。

- 其中，打开 “enable timer driver”，内核会编译并启用定时器驱动程序，允许系统使用该定时器模块。
- 打开 “enable timer HAL APIS test command”，启用与定时器相关的硬件抽象层（HAL）API 的测试功能。HAL（Hardware Abstraction Layer）API 提供了一组标准化的函数，用于操作硬件设备，如定时器、GPIO 等。

配置路径为：Drivers Options > soc related device drivers > enable timer driver。

图 2-1: Timer 模块配置界面



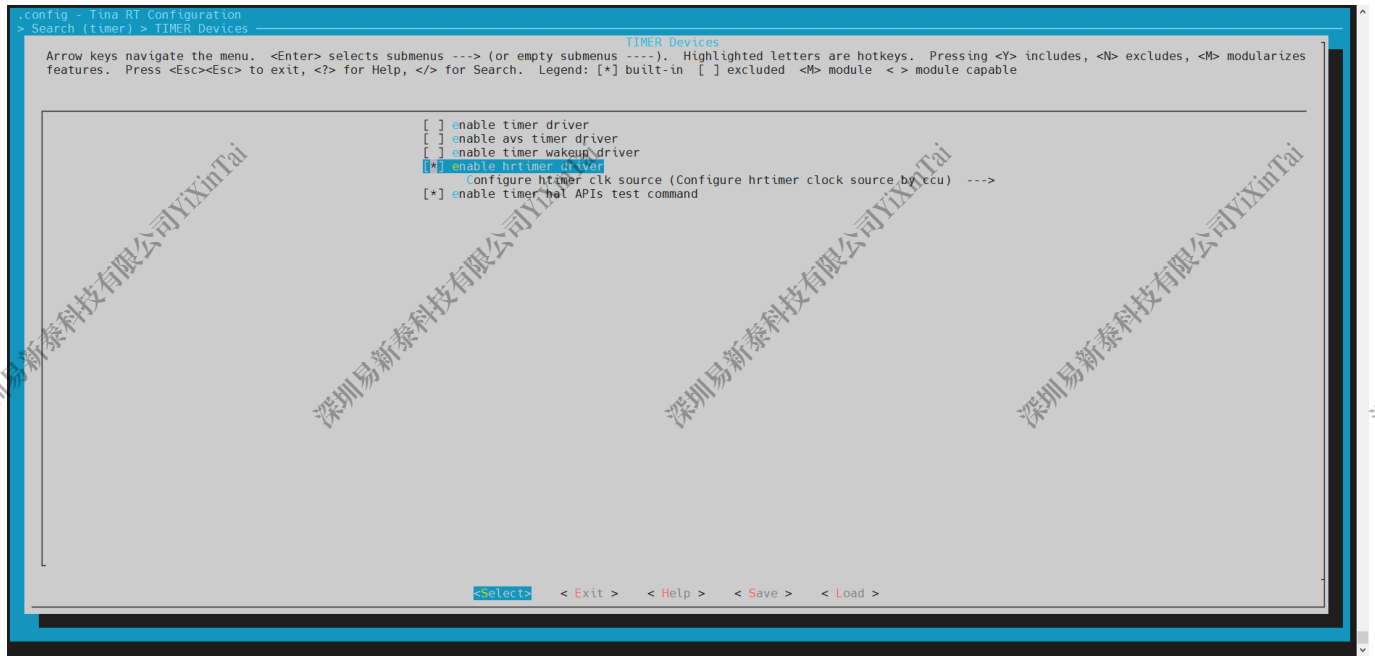
2.3.2 HSTIMER 模块

需要打开 “CONFIG_DRIVERS_HTIMER” 宏定义。

- 其中，打开 “enable hrtimer driver”，启用高精度定时器（Hrtimer），适用于需要高精度时间管理的应用程序或内核驱动。
- 打开 “enable timer HAL APIs test command”，启用与定时器相关的硬件抽象层（HAL）API，并进行测试。

配置路径为：Drivers Options > soc related device drivers > enable hrtimer driver。

图 2-2: HSTimer 模块配置界面



2.4 模块源码结构

2.4.1 TIMER 模块源码结构

Timer 模块源码结构如下所示：

```
rtos-hal/
|--hal/source/timer/hal_timer.c //hal层接口代码
|--hal/source/timer/sunxi_timer.c //sunxi层接口代码
|--hal/source/timer/sunxi_timer.h //sunxi层头文件
|--include/hal/sunxi_hal_timer.h //头文件
```

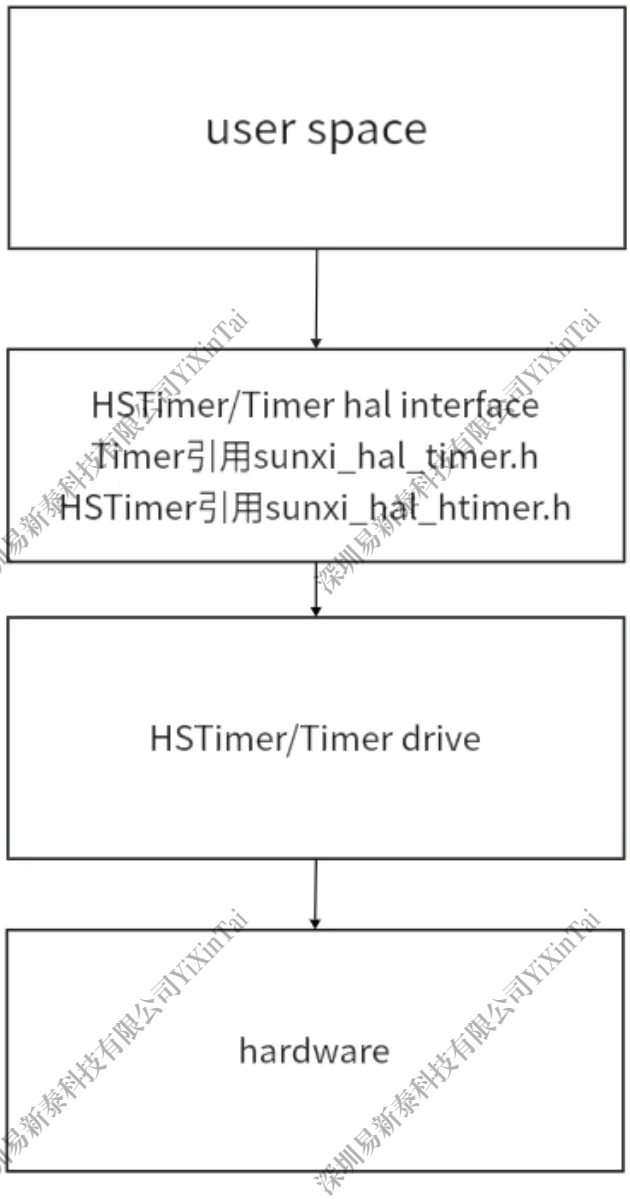
2.4.2 HSTIMER 模块源码结构

```
rtos-hal/
|--hal/source/timer/hal_htimer.c //hal层接口代码
|--hal/source/timer/sunxi_htimer.c //sunxi层接口代码
|--hal/source/timer/sunxi_htimer.h //sunxi层头文件
|--include/hal/sunxi_hal_htimer.h //头文件
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.5 软件框架

图 2-3: 软件框架图



主要软件框架如上图所示。用户层通过调用 sunxi_hal_timer.h（适用于普通定时器场景）和 sunxi_hal_hstimer.h（适用于高精度定时器场景），即可使用对应的 Timer/HSTimer 接口进行开发。而 hal_timer.c 和 hal_hstimer.c 文件分别实现了普通定时器和高精度定时器的驱动功能，这些驱动代码向下适配硬层，向上层提供 API 接口，形成了从硬件到应用的完整功能通路。

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

第 3 章 模块接口说明

3.1 接口列表

3.1.1 TIMER 接口

Timer 提供的主要接口列表如下, 使用这些接口需要引入 sunxi_hal_timer.h 文件。

```
void hal_timer_init(hal_timer_id_t timer);
void hal_timer_uninit(hal_timer_id_t timer);
void hal_timer_stop(hal_timer_id_t timer);
void hal_timer_start(hal_timer_id_t timer, bool periodic);
hal_timer_status_t hal_timer_set_oneshot(hal_timer_id_t timer, uint32_t delay_us, timer_callback callback, void *callback_param);
hal_timer_status_t hal_timer_set_periodic(hal_timer_id_t timer, uint32_t delay_us, timer_callback callback, void *callback_param)
    ;
```

3.1.2 HSTIMER 接口

HSTimer 提供的主要接口列表如下，使用这些接口需要引入 sunxi_hal_htimer.h 文件。

```
void hal_htimer_init(int id);
void hal_htimer_stop(hal_htimer_id timer);
void hal_htimer_start(hal_htimer_id timer, bool periodic);
hal_htimer_status_t hal_htimer_set_oneshot(hal_htimer_id timer, uint32_t delay_us, timer_callback callback, void *callback_param)
    ;
hal_htimer_status_t hal_htimer_set_periodic(hal_htimer_id timer, uint32_t delay_us, timer_callback callback, void *callback_param
    );
void hal_htimer_base_init(int id);/* Enable clock */
void hal_htimer_base_deinit(int id)/* Disable clock */
uint64_t hal_htimer_get_counter(uint32_t timer) //获取当前计数count值。
```

3.2 接口使用说明

以下接口说明兼容 Timer 与 HSTimer，只有函数名称不一致，参数及返回值均一致。

3.2.1 Timer 初始化接口

- 原型：void hal_timer_init(hal_timer_id_t timer)
- 功能：Timer 模块初始化，主要初始化一些工作模式，时钟等等
- 参数：timer，要操作的 Timer 序号
- 返回值：无

3.2.2 Timer 失能接口

- 原型：void hal_timer_uninit(hal_timer_id_t timer)
- 功能：Timer 模块失能，主要失能一些工作模式，时钟等等
- 参数：timer，要操作的 Timer 序号
- 返回值：无

3.2.3 Timer 开启计数接口

- 原型：void hal_timer_start(hal_timer_id_t timer, bool periodic)
- 功能：Timer 模块开启计数
- 参数：timer，要操作的 Timer 序号；periodic，计数周期
- 返回值：无

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

3.2.4 Timer 关闭计数接口

- 原型：void hal_timer_stop(hal_timer_id_t timer, bool periodic)
- 功能：Timer 模块关闭计数
- 参数：timer，要操作的 Timer 序号
- 返回值：无

3.2.5 Timer 周期性定时功能

- 原型：hal_timer_status_t hal_timer_set_periodic(hal_timer_id_t timer, uint32_t delay_us, timer_callback callback, void *callback_param);
- 功能：设置定时功能，延时 delay_us us 后响应中断，周期性触发
- 参数：
  - hal_timer_id_t timer：Timer 的 id 值，表示使用哪个 Timer
  - uint32_t delay_us：延时时间，单位：us
  - timer_callback callback：中断回调函数
  - void *callback_param：中断函数传入的参数
- 返回值：HAL_TIMER_STATUS_OK：设置成功；HAL_TIMER_STATUS_ERROR 设置失败

3.2.6 Timer 单次定时功能

- 原型：hal_timer_status_t hal_timer_set_oneshot(hal_timer_id_t timer, uint32_t delay_us, timer_callback callback, void *callback_param);
- 功能：设置定时功能，延时 delay_us us 后响应中断，只触发一次
- 参数：
  - hal_timer_id_t timer：Timer 的 id 值，表示使用哪个 Timer
  - uint32_t delay_us：延时时间，us
  - timer_callback callback：中断回调函数
  - void *callback_param：中断函数传入的参数
- 返回值：HAL_TIMER_STATUS_OK：设置成功；HAL_TIMER_STATUS_ERROR 设置失败

3.2.7 HSTimer 基础时钟初始化接口

- 原型：void hal_htimer_base_init(int id)
- 功能：启用硬件定时器的时钟源，解除复位状态，使定时器基础功能可运行
- 参数：id，定时器编号
- 返回值：无

3.2.8 HSTimer 基础时钟反初始化接口

- 原型：void hal_htimer_base_deinit(int id)
- 功能：关闭硬件定时器的时钟源，断言复位状态，释放时钟资源
- 参数：id，定时器编号
- 返回值：无

3.2.9 HSTimer 计数器读取接口

- 原型：uint64_t hal_htimer_get_counter(uint32_t timer)
- 功能：读取硬件定时器的当前计数器值（Count Value）
- 参数：timer，定时器编号
- 返回值：`uint64_t`：当前计数器的值（单位：时钟周期数）

版权所有 © 珠海全志科技股份有限公司。保留一切权利

第 4 章 模块使用范例

4.1 TIMER 模块使用范例

1. 设置 Timer 为单次触发模式，并填写回调函数，定时时间到后触发回调函数 demo

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <hal_log.h>
#include <hal_cmd.h>
#include <hal_thread.h>
#include <hal_timer.h>
#include <hal_gpio.h>
#include <sunxi_hal_timer.h>

static void hal_timer_irq_callback(void *param)
{
    printf("timer interrupt!!\n");
    hal_timer_stop(timer); /* disable timer */
    hal_timer_uninit(timer); /* free the irq */
}

int cmd_test_hal_timer(int argc, char **argv)
{
    int timer = SUNXI_TMR0;
    int delay_ms = 5 * 1000;

    if (argc >= 2) {
        if (strcmp("-h", argv[1]) == 0) {
            printf("usage: hal_timer <timer> <delay_ms>\n");
            return 0;
        }
    }

    if (atoi(argv[1]) < SUNXI_TMR_NUM)
        timer = SUNXI_TMR0 + atoi(argv[1]);

    if (argc >= 3)
        delay_ms = atoi(argv[2]);

    printf("test timer%d, wait %fs for interrupt\n", timer, delay_ms/1000.0);
    hal_timer_init(timer);//初始化某个timer
    hal_timer_set_oneshot(timer, delay_ms * 1000, hal_timer_irq_callback, NULL);//设置Timer的单次触发模式，并填写回调函数。
    return 0;
}
```

2. 设置 Timer 为周期模式，并填写回调函数，定时周期到后触发回调函数 demo

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <hal_log.h>
#include <hal_cmd.h>
#include <hal_thread.h>
#include <hal_timer.h>
#include <hal_gpio.h>
#include <sunxi_hal_timer.h>

static void hal_timer_irq_callback(void *param)
{
    printf("timer interrupt!!\n");
}

int cmd_test_hal_timer_periodic_mode(int argc, char **argv)
{
```

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

```
    int timer = SUNXI_TMR0;
    int delay_ms = 5 * 1000;
    int test_time_s = 30;
    if (argc >= 2) {
        if (strcmp("-h", argv[1]) == 0) {
            printf("usage: hal_timer_periodic_mode  <timer> <delay_ms> <test_time_s>\n");
            return 0;
        }
    }

    if (atoi(argv[1]) < SUNXI_TMR_NUM)
        timer = SUNXI_TMR0 + atoi(argv[1]);

    if (argc >= 3)
        delay_ms = atoi(argv[2]);
    if (argc >= 4)
        test_time_s = atoi(argv[3]);
    printf("test timer%d, wait %fs for interrupt\n", timer, delay_ms/1000.0);
    hal_timer_init(timer); /* enable timer and apply irq */
    hal_timer_set_periodic(timer, delay_ms * 1000, hal_timer_irq_callback, NULL);
    sleep(test_time_s);
    printf("after %ds,close the timer%d\n", test_time_s,timer);
    hal_timer_stop(timer); /* disable timer */
    hal_timer_uninit(timer); /* free the irq */
    return 0;
}
```

4.2 HSTIMER 模块使用范例

1. 设置 HSTimer 为单次触发模式，并填写回调函数，定时时间到后触发回调函数 demo

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <hal_log.h>
#include <hal_cmd.h>
#include <hal_thread.h>
#include <hal_timer.h>
#include <hal_gpio.h>
#include <sunxi_hal_htimer.h>

#define HTIMER_IDLE (0)
#define HTIMER_STOP (1)
#define HTIMER_START    (2)
static int timer_irq_flag = HTIMER_IDLE;
static int test_htimer = HAL_HRTIMER0;
static int test_time_s = 30;

static void hal_htimer_irq_callback(void *param)
{
    timer_irq_flag = HTIMER_STOP;
}

void hal_htimer_thread(void *param)
{
    while(1) {
        if (timer_irq_flag == HTIMER_STOP) {
            hal_htimer_deinit(test_htimer); /* Disable htimer and free the irq */
            hal_htimer_base_deinit(test_htimer); /* Disable clock */
            printf("htimer interrupt!!\n");
            break;
        }
        hal_msleep(1);
    }
    timer_irq_flag = HTIMER_IDLE;
    hal_thread_stop(NULL);
}
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
void hal_htimer_periodic_thread(void *param)
{
    int ms_cnt = 0;
    while(1) {
        if (timer_irq_flag == HTIMER_STOP) {
            printf("htimer interrupt!!\n");
            timer_irq_flag = HTIMER_IDLE;
        }
        hal_msleep(1);
        if (++ms_cnt > test_time_s * 1000) {
            hal_htimer_deinit(test_htimer); /* Disable htimer and free the irq */
            hal_htimer_base_deinit(test_htimer); /* Disable clock */
            break;
        }
    }
    timer_irq_flag = HTIMER_IDLE;
    hal_thread_stop(NULL);
}

/* Example of setting a htimer in onshot mode */
int cmd_test_hal_htimer(int argc, char **argv)
{
    int ret;
    int delay_ms = 5 * 1000;
    void *thread;

    if (argc >= 2) {
        if (strcmp("-h", argv[1]) == 0) {
            printf("usage: hal_htimer <timer> <delay_ms>\n");
            return 0;
        }
    }

    if (atoi(argv[1]) < HTIMER_MAX_NUM)
        test_htimer = HAL_HRTIMER0 + atoi(argv[1]);
    else {
        printf("use htimer err, max htimer %d\n", (HTIMER_MAX_NUM - 1));
        return 0;
    }

    if (argc >= 3)
        delay_ms = atoi(argv[2]);

    if (timer_irq_flag != HTIMER_IDLE) {
        printf("htimer busy, timer_irq_flag = 0x%x\n", timer_irq_flag);
        return 0;
    }

    timer_irq_flag = HTIMER_START;
    printf("test htimer%d, wait %fs for interrupt\n", test_htimer, delay_ms/1000.0);

    hal_htimer_base_init(test_htimer); /* Enable clock */

    hal_htimer_init(test_htimer); /* Enable htimer and apply irq */

    /* Set and start one-shot mode */
    ret = hal_htimer_set_oneshot(test_htimer, delay_ms * 1000, hal_htimer_irq_callback, NULL);

    if (ret)
        printf("hal_htimer_set_oneshot fail!!\n");
    else
        printf("hal_htimer_set_oneshot OK!!\n");

    thread = hal_thread_create(hal_htimer_thread, NULL, \
                "hal htimer thread", 512, 29);
    if (thread != NULL)
        hal_thread_start(thread);

    return 0;
}
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

2. 设置 HSTimer 为周期模式，并填写回调函数，定时周期到后触发回调函数 demo

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <hal_log.h>
#include <hal_cmd.h>
#include <hal_thread.h>
#include <hal_timer.h>
#include <hal_gpio.h>
#include <sunxi_hal_htimer.h>

#define HTIMER_IDLE (0)
#define HTIMER_STOP (1)
#define HTIMER_START    (2)
static int timer_irq_flag = HTIMER_IDLE;
static int test_htimer = HAL_HRTIMER0;
static int test_time_s = 30;

static void hal_htimer_irq_callback(void *param)
{
    timer_irq_flag = HTIMER_STOP;
}

void hal_htimer_thread(void *param)
{
    while(1) {
        if (timer_irq_flag == HTIMER_STOP) {
            hal_htimer_deinit(test_htimer); /* Disable htimer and free the irq */
            hal_htimer_base_deinit(test_htimer); /* Disable clock */
            printf("htimer interrupt!!\n");
            break;
        }
        hal_msleep(1);
    }
    timer_irq_flag = HTIMER_IDLE;
    hal_thread_stop(NULL);
}

/* Example of setting a htimer in continuous mode */
int cmd_test_hal_htimer_periodic_mode(int argc, char **argv)
{
    int delay_ms = 5 * 1000;
    void *thread;

    test_time_s = 30;

    if (argc >= 2) {
        if (strcmp("-h", argv[1]) == 0) {
            printf("usage: hal_htimer_periodic_mode  <timer> <delay_ms> <test_time_s>\n");
            return 0;
        }
    }

    if (atoi(argv[1]) < HTIMER_MAX_NUM)
        test_htimer = HAL_HRTIMER0 + atoi(argv[1]);
    else {
        printf("use htimer err, max htimer %d\n", (HTIMER_MAX_NUM - 1));
        return 0;
    }

    if (argc >= 3)
        delay_ms = atoi(argv[2]);

    if (argc >= 4)
        test_time_s = atoi(argv[3]);

    printf("test time = %d s\n", test_time_s);
    printf("test htimer%d, wait %fs for interrupt\n", test_htimer, delay_ms/1000.0);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

```
    hal_htimer_base_init(test_htimer); /* Enable clock */
    hal_htimer_init(test_htimer); /* Enable htimer and apply irq */

    /* Set and start continuous mode */
    hal_htimer_set_periodic(test_htimer, delay_ms * 1000, hal_htimer_irq_callback, NULL);

    thread = hal_thread_create(hal_htimer_periodic_thread, NULL, \
                    "hal htimer periodic thread", 512, 29);
    if (thread != NULL)
        hal_thread_start(thread);

    return 0;
}
```

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

文档问题反馈: aw-document@allwinnertech.com

第 5 章 调试方法

5.1 TIMER 测试方法

1. 开启测试 config：`HAL_TEST_TIMER`
2. 可在命令行输入`hal_timer x`查看当前 Timer 是否正常工作，其中`x`表示需要测试哪一路 TIMER。

5.2 HSTIMER 测试方法

1. 开启测试 config：`HAL_TEST_TIMER`
2. 可在命令行输入`hal_htimer x`查看当前 Timer 是否正常工作，其中`x`表示需要测试哪一路 HSTIMER。

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

第 6 章 FAQ

深圳易新泰科技有限公司YiXinTai

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2026 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。